

IA-AGENT - Guia detallada por etapas

Este documento explica las 8 etapas del framework IA-AGENT y detalla que carpetas y archivos participan en cada una. La base esta orientada solo a agentes con IA agentica. Los casos puramente generativos deben manejarse en una base separada, IA-GEN.

Vista general de carpetas

- ``data/``: contiene los datasets CSV de entrada. Aquí viven los casos de prueba que el framework ejecuta.
- ``config.py``: centraliza carga de ambiente, validación de variables y selección de adapter/perfil.
- ``.env.<ambiente>``: archivos locales de configuración por ambiente, por ejemplo ``.env.desa``. No se versionan.
- ``main.py``: punto de entrada de ejecución. Carga CSV, ejecuta escenarios y dispara reportes.
- ``core/``: núcleo del framework. Orquesta escenarios, contratos, runner, utilidades y trazas.
- ``adapters/``: integraciones concretas con agentes reales. Actualmente ``phoenix`` y ``agentic_rest``.
- ``integrations/``: clientes reutilizables para servicios externos, principalmente Azure OpenAI.
- ``evaluation/``: jueces LLM, perfiles de evaluación, métricas y validaciones.
- ``reporting/``: generación de reportes HTML/CSV a partir de los resultados.
- ``resultados/``: carpeta de salida de reportes y evidencias generadas.
- ``tests/``: pruebas unitarias del runner, adapters, humo de evaluación y contratos principales.
- ``diagramas/``: imágenes generadas para documentar la arquitectura.
- ``tools/``: scripts auxiliares para generar diagramas o documentos.

Etapa 1. Diseño del Dataset de Evaluación

Objetivo

Definir que se va a probar antes de ejecutar el agente. Esta etapa convierte una necesidad de negocio o técnica en casos de prueba estructurados que el framework puede leer.

Carpetas y archivos involucrados

- ``data/casos_de_prueba_desa.csv``: dataset principal para Phoenix/cobranzas.
- ``data/agentes_agenticos.csv``: dataset generico para agentes IA-AGENT expuestos por REST.
- ``data/README.md``: documentacion local del dataset, si se desea ampliar reglas de columnas.
- ``config.py``: consume ``CSV_PATH`` y ``CSV_SEP``, por lo tanto condiciona que dataset se cargara.
- ``evaluation/juez_funcionalidades.py``: define criterios especializados cuando el perfil usa funcionalidades, por ejemplo Phoenix.
- ``evaluation/juez_respuesta.py``: define el juez generico de respuesta para ``agentic_default``.
- ``evaluation/juez_metricas.py``: define metricas reutilizables para cualquier adapter.

Actividades

- Definir el objetivo del caso de prueba: que comportamiento agentic se quiere validar.
- Asignar un identificador estable en ``id_test``.
- Redactar el ``caso_de_prueba`` con el comportamiento esperado.
- Definir ``mensaje_inicio``, que sera el primer mensaje enviado al agente.
- Definir ``secuencia_mensaje`` cuando se necesiten turnos controlados desde CSV.
- Definir ``reglas_negocio_juez`` para indicar al juez que debe considerar PASS, WARNING o FAIL.
- Definir ``ejecutar_prueba=1`` solo para los escenarios que deben correr.
- Para Phoenix, completar datos de cliente, deuda, segmentacion y perfil de simulacion cuando aplique.
- Para ``agentic_rest``, mantener el dataset mas generico y depender de ``metadata`` para campos propios del agente.

Campos recomendados

- ``id_test``: codigo unico del caso.
- ``caso_de_prueba``: descripcion del escenario y comportamiento esperado.
- ``mensaje_inicio``: primer mensaje enviado al agente.
- ``secuencia_mensaje``: mensajes adicionales, uno por linea si hay varios.
- ``ejecutar_prueba``: 1 para ejecutar, 0 para omitir.
- ``reglas_negocio_juez``: criterios que el juez debe usar.
- ``perfil_juez``: perfil funcional opcional para adaptar la evaluacion.
- ``tipo_cliente``: util para Phoenix y escenarios con perfiles de conversacion.

Salida de la etapa

Un CSV valido, versionable y ejecutable por el framework. Este CSV es la fuente de verdad de los escenarios.

Etapa 2. Carga de Configuracion del Framework

Objetivo

Cargar el ambiente activo y validar que la ejecucion tenga todos los parametros necesarios antes de tocar agentes externos o servicios LLM.

Carpetas y archivos involucrados

- `.env.desa`, `.env.certi`, `.env.prod`: archivos de variables por ambiente.
- `config.py`: valida ambiente, adapter, tipo de agente, perfil de evaluacion, Azure OpenAI, CSV, salida y variables REST.
- `requirements.txt`: dependencias necesarias para ejecutar el framework.
- `main.py`: importa la configuracion ya validada.

Variables principales

- `APP_ENV`: ambiente activo. Por defecto se usa `desa`.
- `AGENT_ADAPTER`: adapter a ejecutar. Valores actuales: `phoenix` o `agenticorest`.
- `TIPO_AGENTE`: en esta base debe ser `agenticorest`.
- `EVAL_PROFILE`: perfil de evaluacion. Valores principales: `phoenix_cobranzas` o `agenticorest_default`.
- `CSV_PATH`: ruta del dataset.
- `CSV_SEP`: separador del CSV.
- `OUTPUT_DIR`: carpeta donde se escriben reportes.
- `AZURE_OPENAI_*` y `MODEL_NAME`: configuracion del LLM usado por jueces y simulador Phoenix.
- `AGENTICOREST_*`: configuracion del adapter REST generico.

Actividades

- Resolver `APP_ENV` y buscar `.env.<ambiente>`.
- Cargar variables con `python-dotenv`.
- Validar que `TIPO_AGENTE=agenticorest`.
- Validar que `AGENT_ADAPTER` exista en los adapters permitidos.
- Validar `EVAL_PROFILE` para elegir pipeline de evaluacion.
- Validar variables obligatorias de Azure OpenAI.
- Validar rutas de CSV y carpeta de salida.
- Para Phoenix, exigir `BOT_API_KEY`, `URL_CHAT` y `CUSTOMER_PROC_URL`.
- Para `agenticorest`, leer `AGENTICOREST_URL`, headers, timeout, modo de payload y campo de respuesta.

Salida de la etapa

Constantes de configuracion listas para ser usadas por `main.py`, `core/runner.py`, `adapters/` y `evaluation/`.

Eta­pa 3. Preparacion del Escenario

Objetivo

Convertir cada fila del CSV en un objeto de escenario normalizado, con campos estandar para que el runner no dependa directamente del formato bruto del CSV.

Carpetas y archivos involucrados

- ``main.py``: lee el CSV con pandas y filtra ``ejecutar_prueba == 1``.
- ``core/scenario.py``: define ``Scenario`` y ``scenario_from_row``.
- ``core/utills.py``: normaliza strings y parsea ``secuencia_mensaje``.
- ``core/contracts.py``: define objetos que luego usaran los adapters.
- ``data/``: fuente del escenario.

Actividades

- Leer el CSV usando ``CSV_PATH``, ``CSV_SEP`` y encoding UTF-8.
- Filtrar solo filas habilitadas para ejecucion.
- Convertir cada fila en un diccionario ``metadata``.
- Construir ``Scenario`` con ``id_test``, ``mensaje_inicio``, ``secuencia_mensaje``, ``caso_de_prueba``, ``tipo_cliente``, reglas de negocio y reglas del juez.
- Separar ``simulator_profile`` para Phoenix, sin forzar que otros adapters lo usen.
- Mantener ``metadata`` completa para que cada adapter pueda leer campos propios.

Salida de la etapa

Un Scenario por fila ejecutable. Este objeto es la entrada normalizada del runner.

Etapa 4. Selección e Invocación del Adapter del Agente

Objetivo

Seleccionar que agente real se va a invocar y adaptar el contrato generico del framework al contrato específico de ese agente.

Carpetas y archivos involucrados

- ``adapters/factory.py``: resuelve el adapter desde ``AGENT_ADAPTER``.
- ``adapters/phoenix/``: implementación especializada para Phoenix.
- ``adapters/agentico_rest/``: implementación generica para un agente IA-AGENT por REST.
- ``core/contracts.py``: define ``AgentClient``, ``PreparedScenario``, ``ChatSession`` y ``AgentResponse``.
- ``config.py``: aporta el adapter configurado.

Contrato mínimo del adapter

- ``prepare_scenario(scenario_data)``: transforma metadata del CSV en payload propio del agente.
- ``start_chat(prepared)``: inicia sesión o prepara contexto de conversación.
- ``send_message(session, message)``: envía un mensaje y devuelve una respuesta estándar.
- ``simulate_user(...)``: capacidad opcional. En la arquitectura actual es exclusiva de Phoenix.

Phoenix

- ``adapters/phoenix/payload.py``: construye el payload de cliente.
- ``adapters/phoenix/client.py``: llama customer proc y chat Phoenix.
- ``adapters/phoenix/agent.py``: implementa el contrato ``AgentClient``.
- ``adapters/phoenix/prompts.py``: contiene prompts AR/I/R exclusivos de Phoenix.
- ``adapters/phoenix/simulator.py``: invoca Azure OpenAI para simular cliente.

Agentico REST

- ``adapters/agentico_rest/agent.py``: implementa el contrato para un agente REST generico.
- ``adapters/agentico_rest/client.py``: arma payload HTTP, headers, timeout, parsing de respuesta y ``exit_status``.
- Este adapter no usa ``user_simulator``, prompts AR/I/R ni endpoints Phoenix.

Salida de la etapa

Un cliente de agente listo para ejecutar escenarios bajo una interfaz común.

Eta­pa 5. Ejecucion del Comportamiento Agentico

Objetivo

Ejecutar el comportamiento real del agente: enviar mensajes, recibir respuestas, medir latencias, detectar fin de conversacion y capturar datos crudos.

Carpetas y archivos involucrados

- ``core/runner.py``: orquesta la ejecucion de cada escenario.
- ``core/contracts.py``: estandariza ``AgentResponse``.
- ``adapters/phoenix/client.py``: invoca APIs Phoenix.
- ``adapters/agentico_rest/client.py``: invoca endpoint REST configurado.
- ``integrations/``: participa cuando se requiere LLM, especialmente en simulador/evaluacion.
- ``config.py``: aporta ``MAX_TURNS_SAFE``, timeouts y variables de adapter.

Actividades

- Ejecutar ``prepare_scenario``.
- Iniciar sesion con ``start_chat``.
- Enviar ``mensaje_inicio``.
- Registrar texto del bot, latencia, ``exit_status`` y respuesta cruda.
- Enviar mensajes de ``secuencia_mensaje`` mientras el agente no indique fin.
- Evaluar ``exit_status=1`` como fin de conversacion cuando el adapter lo provee.
- Capturar errores y convertirlos en estado de ejecucion para el reporte.

Diferencia clave entre adapters

- Phoenix puede tener comportamiento agentico mas conversacional y simulacion automatica.
- ``agentico_rest`` ejecuta el contrato REST configurado y normalmente depende de la secuencia definida en CSV.

Salida de la etapa

Una conversacion ejecutada, respuestas normalizadas y datos suficientes para construir trazas y evaluar.

Etapla 6. Conduccion de la Conversacion

Objetivo

Definir como avanza la conversacion despues del mensaje inicial: por secuencia fija desde CSV, por simulador de usuario o por una sola interaccion.

Carpetas y archivos involucrados

- ``core/runner.py``: decide si se usa simulador y controla turnos.
- ``core/utils.py``: arma texto de conversacion para el simulador y para evaluacion.
- ``adapters/phoenix/simulator.py``: ejecuta el simulador de usuario via Azure OpenAI.
- ``adapters/phoenix/prompts.py``: define personalidad/perfil del cliente simulado.
- ``integrations/llm.py``: cliente Azure OpenAI reutilizable.
- ``data/``: aporta ``secuencia_mensaje`` y perfiles de simulacion.

Actividades

- Ejecutar primero la secuencia definida en CSV.
- Si el adapter tiene ``simulate_user``, continuar la conversacion con usuario simulado.
- Aplicar ``MAX_TURNS_SAFE`` para evitar bucles infinitos.
- Terminar cuando el agente indique ``exit_status=1``.
- Terminar si el simulador devuelve fin, adios o texto vacio.
- Para ``agentic_rest``, continuar sin simulador y cerrar con la secuencia definida.

Regla de arquitectura

El `user_simulator` no pertenece al framework completo ni a todos los agentes. En esta base es una capacidad opcional del adapter y actualmente es exclusiva de Phoenix.

Salida de la etapa

Una conversacion completa, con turnos de usuario/agente y latencias acumuladas.

Etapla 7. Construccion de la Traza de Ejecucion

Objetivo

Convertir la ejecucion en evidencia auditable: payload, conversacion, estado, latencias, ultima respuesta y datos para reporte.

Carpetas y archivos involucrados

- ``core/runner.py``: construye cada fila de resultado con ``construir_row_resultado``.
- ``core/utlis.py``: genera conversacion completa con ``build_full_conversation``.
- ``core/contracts.py``: aporta ``ChatSession.raw`` y ``AgentResponse.raw``.
- ``reporting/report.py``: consume las filas para HTML/CSV.
- ``resultados/``: destino final de evidencias generadas.

Datos que se registran

- ``id_test`` y ``chat_id``.
- ``caso_de_prueba`` y perfil del juez.
- ``payload`` enviado o preparado.
- ``conversa``, con turnos de cliente y agente.
- ``answer_last_bot``.
- ``bot_turns``, ``bot_latency_s_total`` y ``sim_latency_s_total``.
- ``status`` tecnico de ejecucion.
- ``secuencia_mensaje``.
- Campos de evaluacion agregados despues por los jueces.

Actividades

- Mantener historial de conversacion en memoria.
- Registrar cada mensaje de usuario y bot.
- Acumular latencias.
- Capturar excepciones como evidencia y no perder el caso completo.
- Convertir objetos internos a JSON seguro para reporte.

Salida de la etapa

Una fila estructurada por escenario, lista para evaluacion y reporte.

Etapa 8. Evaluacion y Generacion de Reportes

Objetivo

Agrupar evaluacion, reportes y evidencias. Esta etapa determina si el agente cumple el caso de prueba, calcula metricas y genera artefactos finales.

Carpetas y archivos involucrados

- ``evaluation/juez.py``: fachada principal que ejecuta el pipeline de evaluacion.
- ``evaluation/profiles/registry.py``: mapea ``EVAL_PROFILE`` a pipeline.
- ``evaluation/juez_funcionalidades.py``: juez especializado para funcionalidades Phoenix/cobranzas.
- ``evaluation/juez_respuesta.py``: juez generico para ``agentic_default``.
- ``evaluation/juez_metricas.py``: juez reutilizable de metricas.
- ``integrations/llm.py``: cliente Azure OpenAI para llamar jueces LLM.
- ``reporting/report.py``: genera HTML y CSV.
- ``resultados/``: guarda ``Rep-paralelizado-<fecha>.html`` y ``*.csv``.

Pipelines actuales

- ``phoenix_cobranzas``: ejecuta ``juez_funcionalidades`` y luego ``juez_metricas``.
- ``agentic_default``: ejecuta ``juez_respuesta`` y luego ``juez_metricas``.

Actividades de evaluacion

- Construir conversacion completa para el juez.
- Aplicar reglas del caso de prueba.
- Ejecutar juez principal segun perfil.
- Ejecutar juez de metricas para coherencia, fluidez, cumplimiento, integridad, claridad y correccion.
- Adjuntar JSON crudo del juez para auditoria.
- Clasificar resultado como PASS, WARNING o FAIL.

Actividades de reporte

- Consolidar resultados en un DataFrame.
- Calcular resumen general.
- Generar tablas, badges, detalle de metricas y modales de evidencia.
- Exportar HTML navegable.
- Exportar CSV con todos los campos para analisis posterior.

Evidencias generadas

- Conversacion completa.
- Payload usado por el adapter.
- JSON del juez principal.
- JSON del juez de metricas.
- Latencias y estados.
- HTML y CSV final.

Salida de la etapa

Reporte final en resultados/, con evaluación funcional o de respuesta, métricas, evidencia y trazabilidad por escenario.

Matriz resumida por etapa

- Etapa 1 usa principalmente ``data/`` y define las reglas que usara ``evaluation/``.
- Etapa 2 usa ``env.<ambiente>`` y ``config.py``.
- Etapa 3 usa ``main.py``, ``core/scenario.py`` y ``core/utils.py``.
- Etapa 4 usa ``adapters/factory.py``, ``adapters/phoenix/``, ``adapters/agentico_rest/`` y ``core/contracts.py``.
- Etapa 5 usa ``core/runner.py``, los clientes dentro de ``adapters/`` y servicios externos.
- Etapa 6 usa ``core/runner.py``, ``adapters/phoenix/simulator.py``, ``adapters/phoenix/prompts.py`` e ``integrations/llm.py`` cuando aplica.
- Etapa 7 usa ``core/runner.py``, ``core/utils.py``, contratos y prepara datos para ``reporting/``.
- Etapa 8 usa ``evaluation/``, ``integrations/llm.py``, ``reporting/`` y ``resultados/``.

Recomendacion para mantener el framework limpio

- Mantener IA-AGENT solo para agentes con comportamiento agentico.
- No mezclar flujos IA-GEN dentro de este repositorio base.
- Tratar cada nuevo agente como un adapter dentro de `adapters/<nombre>`.
- No poner logica de un agente especifico dentro de `core`.
- Mantener `user\_simulator` como capacidad opcional del adapter, no como obligacion del framework.
- Versionar datasets de ejemplo sin datos sensibles.
- No versionar `.env.\*`, tokens ni endpoints privados.